

Para realizar el trabajo práctico se utilizó Redis en Linux, en Windows y Redis online.

Para utilizarlo en Linux, se lo instaló mediante el gestor de paquetes **yay** y posteriormente iniciando el servicio.

Para utilizarlo en Windows se inició un contenedor de Docker con la imagen de redis y luego con **docker exec** se permitió ejecutar los comandos dentro del contenedor.

Para utilizarlo online, simplemente se creo una cuenta desde el sitio de redis

Parte 1: Introducción

1) ¿Qué tipo de bd es Redis?

Redis es una base de datos **en memoria** de tipo **clave-valor** utilizada principalmente para guardar información que requiere ser accedida de manera frecuente y rápida.

2) ¿Dónde almacena los datos Redis?

Redis es un proceso que se ejecuta y almacena la información en memoria RAM.

3) ¿Qué tipos de datos posee Redis?

Redis provee los siguientes tipos de datos:

String: es el tipo más simple, y permite guardar texto u objetos serializados

Hash: son una colección de pares clave-valor, útiles para representar objetos o contadores

List: son listas enlazadas compuestas por elementos string. Se utilizan, por ejemplo, para implementar stacks y queues.

Set: es una colección sin orden de strings **únicos**. Una de las principales ventajas es la de poder insertar, eliminar y buscar en complejidad temporal $O(1)$.

Sorted set: tiene un concepto similar al set, pero los elementos mantienen un orden en base a un puntaje establecido, de manera que un elemento es mayor a otro si el puntaje del primero es mayor al del segundo. Pueden utilizarse, por ejemplo, para implementar una tabla de puntos de un juego.

Vector set: es un tipo de dato agregado recientemente, diseñado para manejar información vectorial multidimensional y con casos de uso en machine learning, sistemas de recomendación y búsqueda semántica.

Stream: es una estructura de datos utilizada principalmente para insertar la información al final de la estructura, permitiendo registrar eventos en el orden en que ocurren.

Bitmap: permiten realizar operaciones bit a bit sobre strings. En particular, más que un tipo de dato pueden considerarse como un conjunto de operaciones orientadas al trabajo con bits definidas sobre un string, el cual es tratado como un vector de bits.

Bitfield: permiten representar múltiples contadores en un string,

Índices Geoespaciales: permiten almacenar coordenadas y poder buscar ubicaciones dentro de límites geográficos

JSON

Tipos de datos probabilísticos: permiten realizar cálculos estadísticos de una manera aproximada y altamente eficiente, haciendo uso de distintas estructuras de datos especializadas

Series temporales: permite almacenar y consultar datos con timestamps.

4) Enuncie las características de Redis

Dentro de las características que pueden comentarse de Redis se encuentran las siguientes:

- Es una base de datos NoSQL
- El almacenamiento se realiza en memoria
- Alto rendimiento, en gran parte por la característica anterior
- Gran variedad de estructuras de datos avanzadas, como las comentadas en el punto anterior
- Soporta replicación con un modelo de servidor principal y secundario

5) Comparar Redis con los RDBMS

	Redis	RDBMS
Tipo de BD	En memoria, no relacional, basada en un formato clave-valor	Relacional
Lugar de almacenamiento	Memoria RAM	Disco (Almacenamiento

		persistente)
Esquema	Permite tener un esquema flexible	Debe definirse el esquema previo al almacenamiento de datos
Forma de almacenamiento	Diversas estructuras de datos	La BD consta de tablas integradas por tuplas. Permite generar vistas de la información.
Forma de consultar	Comandos específicos para cada tipo de dato.	Sentencias SQL (SELECT, INSERT, UPDATE, DELETE), Stored Procedures

6) ¿Redis tiene transacciones?

Redis tiene un soporte para transacciones particular, el cual permite la ejecución de un conjunto de comandos de manera atómica, en un solo paso.

Todos los comandos en la transacción se ejecutan secuencialmente, y nunca se servirá un requerimiento de otro cliente hecho en el medio de la transacción

Con el comando **MULTI** se inicia la transacción. A partir de ese momento, se pueden indicar todos los comandos que se quiere ejecutar atómicamente. Luego, con el comando **EXEC**, todos los comandos son ejecutados de una vez. Si en lugar de EXEC se realiza **DISCARD**, se vaciará la cola de comandos de la transacción y esta finalizará.

Redis no tiene soporte para hacer **rollback**

Con **WATCH** se puede implementar un lockeo optimista, indicando que se quiere “observar” cierta clave y, si su valor intenta ser cambiado por parte de otro cliente entre el WATCH y el EXEC, la transacción abortará. Luego se deberá repetir la operación.

7) ¿Redis tiene persistencia?

Aunque no es su principal característica, Redis ofrece almacenamiento persistente de distintas maneras:

- RDB (Redis Database): Realiza snapshots o capturas del dataset en intervalos especificados.
- AOF (Append Only File): realiza logging de cada operación de escritura que recibe el servidor. Las operaciones pueden reproducirse luego al reiniciar Redis para reconstruir el dataset.
- Sin persistencia: utilizado para hacer caching
- RDB y AOF combinados: se aprovechan las ventajas de ambos, la performance de RDB, con la seguridad de AOF frente a eventos inesperados.

8) ¿Cuáles son los principales usos de Redis?

Sus usos se centran mayormente en la implementación de caches y sesiones, sistemas basados en tiempo real, debido a su gran rendimiento y variedad de estructuras de datos o almacenamiento temporal, como un historial de búsqueda al loguearse, creando un índice de búsqueda.

Parte 2: Manejo Simple de Valores String

1) Agregue una clave package con el valor "Bariloche 3 days"

```
127.0.0.1:6379> SET package "Bariloche 3 days"
OK
```

2) Agregue una clave user con el valor "Turismo BD2". Obtenga el valor de la clave user

```
127.0.0.1:6379> SET user "Turismo BD2"
OK
127.0.0.1:6379> GET user
"Turismo BD2"
```

3) Obtenga todos los valores de claves almacenadas

```
127.0.0.1:6379> KEYS *
1) "user"
2) "package"
```

4) Agregue una clave user con el valor "Cronos Turismo" ¿Cuál es el valor actual de la clave user?

```
127.0.0.1:6379> SET user "Cronos Turismo"
OK
127.0.0.1:6379> GET user
"Cronos Turismo"
```

Se sobrescribe el valor que poseía la clave previamente

5) Concatene " S.A." a la clave user. ¿Cuál es el valor actual de la clave user?

```
127.0.0.1:6379> APPEND user " S.A."  
(integer) 19  
127.0.0.1:6379> GET user  
"Cronos Turismo S.A."
```

APPEND retorna la nueva longitud del string.

6) Elimine la clave user

```
127.0.0.1:6379> DEL user
```

7)¿Qué valor retorna si queremos obtener la clave user?

```
127.0.0.1:6379> GET user  
(nil)
```

Parte 3: Manejo Simple de Valores Numéricos

1) Verificar si existe la clave visits

```
127.0.0.1:6379> EXISTS visits  
(integer) 0
```

No existe (longitud cero)

2) Agregue una clave visits con el valor 0

```
127.0.0.1:6379> SET VISITS 0
```

3) Incremente en 1 visits ¿Cuál es el valor actual de la clave visits?

```
127.0.0.1:6379> INCR visits  
(integer) 1  
127.0.0.1:6379> GET visits  
"1"
```

Luego del incremento, el valor de la clave visits es 1

4) Incremente en 5 visits. ¿Cuál es el valor actual de la clave visits?

```
127.0.0.1:6379> INCRBY visits 5  
(integer) 6  
127.0.0.1:6379> GET visits  
"6"
```

Ahora tiene el valor 6

5) Decremento en 1 visits ¿Cuál es el valor actual de la clave visits?

```
127.0.0.1:6379> DECR visits  
(integer) 5
```

Ahora vuelve a tener el valor 5

6) Incremente en 2 visits. ¿Cuál es el valor actual de la clave visits?

```
127.0.0.1:6379> INCRBY visits 2  
(integer) 7
```

Ahora tiene el valor 7

7) Agregue una clave "value package" con el valor 539789.32

```
127.0.0.1:6379> SET "value package" 539789.32
```

8) Incremente en 20000 la clave "value package". ¿Cuál es el valor actual de "value package"?

```
127.0.0.1:6379> INCRBYFLOAT "value package" 20000  
"559789.31999999999999999318"
```

Si se utiliza INCRBY, se obtiene un error, dado que se debe incrementar el string representando a un número teniendo en cuenta que es un número con decimales.

9) ¿Cuál es el tipo de datos de "value package", visits y user?

```
127.0.0.1:6379> TYPE "value package"
string
127.0.0.1:6379> TYPE visits
string
127.0.0.1:6379> TYPE user
none
```

Por más que se manejen como números, las claves guardan valores de tipo string. Al preguntar por el tipo de la clave user se obtiene none, dado que esta clave fue previamente borrada. Si se volviese a crear con los mismos valores anteriores, también sería **string**.

```
127.0.0.1:6379> SET user "Turismo BD2"
OK
127.0.0.1:6379> TYPE user
string
```

Parte 4: Manejo de Claves

1) Obtenga todas las claves que empiecen con v

```
127.0.0.1:6379> KEYS v*
1) "value package"
2) "visits"
```

2) Obtenga todas las claves que contengan la "t"

```
127.0.0.1:6379> KEYS *t*
1) "visits"
```

3) Obtenga todas las claves que terminen con "age"

```
127.0.0.1:6379> KEYS *age
1) "value package"
2) "package"
```

4) Renombre la clave “package” por “bariloche package”

```
127.0.0.1:6379> RENAME package "bariloche package"
```

5) Si quisiera evitar renombrar una clave por otra existente, ¿qué comando utilizaría?

Si se quiere renombrar una clave evitando ponerle el nombre de otra clave existente, se puede utilizar RENAMENX, el cual renombra la clave al nombre especificado siempre y cuando no exista una clave con dicho nombre. Si el nombre de clave nuevo ya existe, se retorna 0, caso contrario 1

6) Elimine todas las claves

```
127.0.0.1:6379> FLUSHDB
OK
127.0.0.1:6379> KEYS *
(empty array)
```

Con FLUSHDB se eliminan todas las keys de la base de datos actual. Con FLUSHALL se pueden eliminar todas las keys de todas las bases de datos.

Parte 5: Expiración de Claves

1) Agregue una clave agency con el valor “Cronos Tours”

```
SET agency "Cronos Tours"
```

```
"OK"
```

2) ¿Cuál es el tiempo de vida de la clave agency?

```
TTL agency
```

```
(integer) -1
```

-1 significa que la clave existe pero no tiene expiración definida.

3) Agregue una expiración de 30 segundos a la clave agency

```
EXPIRE agency 30
```

```
(integer) 1
```

4) ¿Cuál es el tiempo de vida de la clave agency?
El tiempo de vida son 30 segundos antes de expirar.

```
TTL agency
```

```
(integer) 25
```

La captura muestra que a la variable le quedan 25 segundos de vida.

5) Pasados los 30 segundos, ¿Cuál es el tiempo de vida de la clave agency? ¿Qué retorna si pido el valor de agency?

```
TTL agency
```

```
(integer) -2
```

```
GET agency
```

```
(nil)
```

La primera captura muestra que el comando “TTL agency” devolvió “-2”, lo que indica que la clave agency ya no existe en la base de datos. En este caso, la causa es que su tiempo de vida (TTL) expiró, por lo tanto Redis la eliminó automáticamente.

La segunda captura muestra que al ejecutar “GET agency”, la respuesta es “nil”, lo cual confirma que la clave agency no está almacenada actualmente.

6) Agreguemos una clave agency con el valor “Cronos Tours” que expire en 20 segundos.

```
SET agency "Cronos Tours" EX 20
```

```
"OK"
```

```
TTL agency
```

```
(integer) 17
```

La primera captura muestra la creación de la clave agency con el valor “Cronos Tours” y le asigna un tiempo de vida de 20 segundos.

La segunda captura muestra el tiempo de vida restante de la clave.

Parte 6: Listas

1) Inserte una lista llamada pets con el valor dog.

```
RPUSH pets dog
```

```
(integer) 1
```

2) ¿Qué sucede si ejecuto el comando get pets? ¿Cómo obtengo los valores de la lista?

```
GET pets
```

```
"WRONGTYPE Operation against a key holding the wrong kind of value"
```

El error ocurre porque el comando “GET” sirve para obtener valores de claves de tipo string y pets es una lista.

Para obtener los valores de la lista correctamente, se debe usar “LRANGE pets 0 -1”, que devuelve todos los elementos de la lista desde el índice 0 hasta el último.

```
LRANGE pets 0 -1
```

```
1) "dog"
```

3) Agregue a la lista pets el valor cat a la izquierda.

```
LPUSH pets cat
```

```
(integer) 2
```

4) Agregue a la lista pets el valor fish a la derecha.

```
RPUSH pets fish
```

```
(integer) 3
```

5) ¿Qué tipo de datos es el valor de pets?

```
TYPE pets
```

```
"list"
```

Como muestra la captura, la clave pets es de tipo lista.

6) Elimine el valor a la izquierda de la lista.

```
LPOP pets
```

```
"cat"
```

7) Elimine el valor a la fecha de la lista.

```
RPOP pets
```

```
"fish"
```

8) Agregue una clave “vuelo:ar389” los valores: aep, mdz, brc, nqn y mdq.

```
RPUSH vuelo:ar389 aep mdz brc nqn mdq
```

```
(integer) 5
```

9) Ordene los valores de lista “vuelo:ar389”. ¿Qué sucede si solicito todos los valores de la lista?

```
SORT vuelo:ar389 ALPHA
```

```
1) "aep"  
2) "brc"  
3) "mdq"  
4) "mdz"  
5) "nqn"
```

```
LRANGE vuelo:ar389 0 -1
```

```
1) "aep"  
2) "mdz"  
3) "brc"  
4) "nqn"  
5) "mdq"
```

El comando “SORT mylist ALPHA” devuelve los valores de la lista ordenados alfabéticamente, pero no altera el orden real de los elementos dentro de la lista. Esto se verifica en la segunda captura, ejecutada luego del SORT, que muestra que el orden original se mantiene intacto.

10) Inserte el valor “fte” luego de “brc”

```
LINSERT vuelo:ar389 AFTER brc fte
```

```
(integer) 6
```

11) Inserte el valor “ush” antes de “fte”

```
LINSERT vuelo:ar389 BEFORE fte ush
```

```
(integer) 7
```

```
127.0.0.1:6379> LRANGE vuelo:ar389 0 -1
1) "aep"
2) "mdz"
3) "brc"
4) "ush"
5) "fte"
6) "nqn"
7) "mdq"
```

12) Modifique el último elemento por “sla”

```
LLEN vuelo:ar389
```

```
LSET vuelo:ar389 6 sla
```

```
(integer) 7
```

```
"OK"
```

Para modificar un elemento de la lista debo saber en qué índice está ubicado.

La primera captura retorna el tamaño de la lista, como los índices de la lista arrancan en 0, el último elemento va a estar en el índice 6.

La segunda captura modifica el último elemento por “sla”.

```
127.0.0.1:6379> LRANGE vuelo:ar389 0 -1
1) "aep"
2) "mdz"
3) "brc"
4) "ush"
5) "fte"
6) "nqn"
7) "sla"
```

13) Obtenga la cantidad de elementos de “vuelo:ar389”

```
LLEN vuelo:ar389
```

```
(integer) 7
```

14) Obtenga el 3 valor de “vuelo:ar389”

```
LINDEX vuelo:ar389 2
```

```
"brc"
```

15) Elimine el valor “aep” de “vuelo:ar389”

```
LREM vuelo:ar389 1 aep
```

```
(integer) 1
```

16) Quédese con los valores de las posiciones 3 a 5 de “vuelo:ar389”

```
LTRIM vuelo:ar389 3 5
```

```
"OK"
```

```
127.0.0.1:6379> LRANGE vuelo:ar389 0 -1
1) "fte"
2) "nqn"
3) "sla"
```

17) Agregue en “vuelo:ar389” el valor “fte”. ¿Cuántas veces aparece?

```
LRANGE vuelo:ar389 0 -1
```

```
RPUSH vuelo:ar389 fte
```

```
(integer) 4
```

```
1) "fte"
2) "nqn"
3) "sla"
4) "fte"
```

El valor “fte” aparece dos veces ya que Redis permite elementos repetidos en listas.

Parte 7: Conjuntos

1) Agregue un conjunto llamado airports los valores: eze aep nqn mdz mdq ush fte sla aep nqn brc cpc juj aep tuc eqs

```
SADD airports eze aep nqn mdz mdq ush fte sla aep nqn brc cpc juj aep tuc eqs
```

```
(integer) 13
```

Los valores duplicados no se agregan.

2) ¿Cuántos valores tiene el conjunto?

```
SCARD airports
```

```
(integer) 13
```

SCARD: Devuelve la cardinalidad del conjunto (número de elementos) almacenado en la clave. Por lo tanto el conjunto airports tiene 13 elementos.

3) Liste los valores del conjunto airports.

```
SMEMBERS airports
```

```
1) "eze"  
2) "aep"  
3) "nqn"  
4) "mdz"  
5) "mdq"  
6) "ush"  
7) "fte"  
8) "sla"  
9) "brc"  
10) "cpc"  
11) "juj"  
12) "tuc"  
13) "eqs"
```

4) Quite el valor cpc del conjunto airports.

```
SREM airports cpc
```

```
(integer) 1
```

5) Quite un valor aleatorio del conjunto airports.

```
SPOP airports
```

```
"fte"
```

6) ¿Qué cantidad de valores tiene ahora airports?

```
SCARD airports
```

```
(integer) 11
```

7) Compruebe si cpc es miembro del conjunto airports.

```
SISMEMBER airports cpc
```

```
(integer) 0
```

8) Mueva los valores sla y juj a un conjunto denominado noa_airports.

```
SMOVE airports noa_airports juj
```

```
(integer) 1
```

```
SMOVE airports noa_airports sla
```

```
(integer) 1
```

9) Retorne la unión de los conjuntos airports y noa_airports. ¿Modifica los conjuntos base?

```
SUNION airports noa_airports
```

- 1) "eze"
- 2) "aep"
- 3) "nqn"
- 4) "mdz"
- 5) "mdq"
- 6) "ush"
- 7) "brc"
- 8) "tuc"
- 9) "eqs"
- 10) "sla"
- 11) "juj"

La unión devuelve un conjunto de elementos únicos a partir de ambos conjuntos, sin modificar ninguno de ellos.

10) Realice la unión de los conjuntos airports y noa_airports en un conjunto llamado total_airports.

```
SUNIONSTORE total_airports airports noa_airports
```

```
(integer) 11
```

11) Realice la intersección entre los conjuntos total_airports y noa_airports.

```
SINTER total_airports noa_airports
```

```
1) "sla"  
2) "juj"
```

12) Realice la diferencia entre los conjuntos total_airports y noa_airports.

```
SDIFF total_airports noa_airports
```

```
1) "eze"  
2) "aep"  
3) "nqn"  
4) "mdz"  
5) "mdq"  
6) "ush"  
7) "brc"  
8) "tuc"  
9) "eqs"
```

Parte 8: Conjuntos Ordenados

1) En un conjunto ordenado llamado passengers agregue los siguientes datos: 2.5 federico 4 alejandra 3 julian 1 ivan 2 andrea 2 luciana 2.4 natalia

```
127.0.0.1:6379> ZADD passengers 2.5 federico 4 alejandra  
3 julian 1 ivan 2 andrea 2 luciana 2.4 natalia  
(integer) 7
```

2) Obtenga los valores del conjunto passengers


```
127.0.0.1:6379> ZRANGE passengers 0 -1
1) "ivan"
2) "andrea"
3) "luciana"
4) "natalia"
5) "federico"
6) "julian"
7) "alejandra"
```

3) Actualice el score de luciana a 2.7

```
127.0.0.1:6379> ZADD passengers 2.7 luciana
(integer) 0
```

4) Agregue al conjunto passengers a silvia con score 5.1

```
127.0.0.1:6379> ZADD passengers 5.1 silvia
(integer) 1
```

5) Incremente en 2 el score de alejandra en el conjunto passengers.

```
127.0.0.1:6379> ZINCRBY passengers 2 alejandra
"6"
```

6) Obtenga los valores del conjunto passengers con sus scores.

```
127.0.0.1:6379> ZRANGE passengers 0 -1 WITHSCORES
1) "ivan"
2) "1"
3) "andrea"
4) "2"
5) "natalia"
6) "2.4"
7) "federico"
8) "2.5"
9) "luciana"
10) "2.7"
11) "julian"
12) "3"
13) "silvia"
14) "5.1"
15) "alejandra"
16) "6"
```

7) Obtenga los valores del conjunto passengers con sus scores en orden inverso.

```
127.0.0.1:6379> ZREVRANGE passengers 0 -1 WITHSCORES
1) "alejandra"
2) "6"
3) "silvia"
4) "5.1"
5) "julian"
6) "3"
7) "luciana"
8) "2.7"
9) "federico"
10) "2.5"
11) "natalia"
12) "2.4"
13) "andrea"
14) "2"
15) "ivan"
16) "1"
```

8) Obtenga la cantidad de elementos del conjunto passengers.

```
127.0.0.1:6379> ZCARD passengers  
(integer) 8
```

9) Obtenga la cantidad de elementos que tienen scores entre 2 y 3.

```
127.0.0.1:6379> ZCOUNT passengers 2 3  
(integer) 5
```

10) Obtenga el ranking de julian en el conjunto passengers.

```
127.0.0.1:6379> ZRANK passengers julian  
(integer) 5
```

11) Obtenga el score de andrea en el conjunto passengers.

```
127.0.0.1:6379> ZSCORE passengers andrea  
"2"
```

12) Extraiga el valor de menor score del conjunto passengers.

```
127.0.0.1:6379> ZPOPMIN passengers  
1) "ivan"  
2) "1"
```

13) Extraiga el valor de mayor score del conjunto passengers.

```
127.0.0.1:6379> ZPOPMAX passengers  
1) "alejandra"  
2) "6"
```

14) Elimine del conjunto passengers al valor silvia.

```
127.0.0.1:6379> ZREM passengers silvia  
(integer) 1
```

Parte 9: Hashes

1) Agregue a un hash llamado user:cronos los valores: "razon social" "cronos s.a", domicilio "47 236 La Plata", "teléfono" 2215556677

```
127.0.0.1:6379> HSET user:cronos "razon social" "cronos s.a"  
domicilio "47 236 La Plata" teléfono 2215556677  
(integer) 3
```

2) Agregue el mail info@cronos.com.ar a user:cronos

```
127.0.0.1:6379> HSET user:cronos mail "info@cronos.com.ar"  
(integer) 1
```

3) Obtenga todos los valores de user:cronos

```
127.0.0.1:6379> HGETALL user:cronos  
1) "razon social"  
2) "cronos s.a"  
3) "domicilio"  
4) "47 236 La Plata"  
5) "telefono"  
6) "2215556677"  
7) "mail"  
8) "info@cronos.com.ar"  
127.0.0.1:6379> █
```

4) Obtenga el mail de user:cronos

```
127.0.0.1:6379> HGET user:cronos mail  
"info@cronos.com.ar"
```

5) Elimine el teléfono de user:cronos

```
127.0.0.1:6379> HDEL user:cronos teléfono  
(integer) 1
```

6) Obtenga la cantidad de campos de user:cronos

```
127.0.0.1:6379> HLEN user:cronos  
(integer) 3
```

7) Obtenga las claves de los campos de user:cronos

```
127.0.0.1:6379> HKEYS user:cronos  
1) "razon social"  
2) "domicilio"  
3) "mail"
```

8) Determine si existe el campo cuil en user:cronos

```
127.0.0.1:6379> HEXISTS user:cronos cuil  
(integer) 0
```

9) Obtenga todos los valores de los campos de user:cronos

```
127.0.0.1:6379> HVALS user:cronos  
1) "cronos s.a"  
2) "47 236 La Plata"  
3) "info@cronos.com.ar"
```

10) Obtenga la longitud del campo mail de user:cronos

```
127.0.0.1:6379> HSTRLEN user:cronos mail  
(integer) 18
```

Parte 10: Geospacial

1. Agregue en un conjunto denominado cities las siguientes localidades:

Buenos Aires	-34.61315, -58.37723
Córdoba	-31.4135, -64.18105
Rosario	-32.94682, -60.63932
Mendoza	-32.89084, -68.82717

San Miguel de Tucumán	-26.82414, -65.2226
La Plata	-34.92145, -57.95453
Mar del Plata	-38.00042, -57.5562
Salta	-24.7859, -65.41166
Santa Fe	-31.64881, -60.70868
San Juan	-31.5375, -68.53639
Resistencia	-27.46056, -58.98389
Santiago del Estero	-27.79511, -64.26149
Posadas	-27.36708, -55.89608
San Salvador de Jujuy	-24.19457, -65.29712
Bahía Blanca	-38.71959, -62.27243
Paraná	-31.73271, -60.52897

```
127.0.0.1:6379> GEOADD cities -58.37723 -34.61315 "Buenos Aires" -
64.18105 -31.4135 "Córdoba" -60.63932 -32.94682 "Rosario" -68.8271
7 -32.89084 "Mendoza" -65.2226 -26.82414 "San Miguel de Tucumán" -
57.95453 -34.92145 "La Plata" -57.5562 -38.00042 "Mar del Plata" -
65.41166 -24.7859 "Salta" -60.70868 -31.64881 "Santa Fe" -68.53639
-31.5375 "San Juan" -58.98389 -27.46056 "Resistencia" -64.26149 -
27.79511 "Santiago del Estero" -55.89608 -27.36708 "Posadas" -65.2
9712 -24.19457 "San Salvador de Jujuy" -62.27243 -38.71959 "Bahía
Blanca" -60.52897 -31.73271 "Paraná"
(integer) 16
```

2) Obtenga los valores del conjunto cities

```
127.0.0.1:6379> ZRANGE cities 0 -1
1) "Mendoza"
2) "San Juan"
3) "Bah\xca3\xada Blanca"
4) "Mar del Plata"
5) "Buenos Aires"
6) "La Plata"
7) "Rosario"
8) "C\xca3\xb3rdoba"
9) "San Miguel de Tucum\xca3\x1n"
10) "Santiago del Estero"
11) "Paran\xca3\x1"
12) "Santa Fe"
13) "Resistencia"
14) "Salta"
15) "San Salvador de Jujuy"
16) "Posadas"
```

3) Obtenga las coordenadas de Santa Fe

```
127.0.0.1:6379> GEOPOS cities "Santa Fe"
1) 1) "-60.70868164300918579"
   2) "-31.64881101691540977"
```

4) Obtenga la distancia en km entre Buenos Aires y Córdoba

```
127.0.0.1:6379> GEODIST cities "Buenos Aires" "Córdoba" km
"647.6303"
```

5) Obtenga las ciudades que están en un radio de 100 km de la coordenada -27.37 -55.9 con su distancia.

```
127.0.0.1:6379> GEORADIUS cities -55.9 -27.37 100 km WITHDIST
1) 1) "Posadas"
   2) "0.5055"
```

6) Obtenga las ciudades que están a menos de 700 km de Córdoba.

```
127.0.0.1:6379> GEORADIUSBYMEMBER cities "Córdoba" 700 km WITHDIST
1) 1) "Córdoba"
   2) "0.0000"
2) 1) "San Miguel de Tucumán"
   2) "520.3835"
3) 1) "Santiago del Estero"
   2) "402.5353"
4) 1) "Paraná"
   2) "347.8771"
5) 1) "Santa Fe"
   2) "330.2202"
6) 1) "Resistencia"
   2) "668.2173"
7) 1) "Buenos Aires"
   2) "647.6303"
8) 1) "La Plata"
   2) "698.5502"
9) 1) "Rosario"
   2) "374.4698"
10) 1) "San Juan"
    2) "413.3545"
11) 1) "Mendoza"
    2) "467.3004"
```